Conceptual Exploration of Equational Theories of Magmas

Implementation notes, two case studies, and a user's guide

The `conceptual-exploration` library

October 2026

Abstract

This document describes how the `conceptual-exploration` library carries out the attribute exploration of implications between equational laws of magmas, the question studied by the Equational Theories Project. It is written for a reader who knows attribute exploration as presented in Ganter and Obiedkov's *Conceptual Exploration*, and it concentrates on what is done differently from the book: partial objects, symmetries, experts that cannot always decide, and the resumption of an exploration when an implication accepted without proof is refuted. It then describes what is specific to equational theories: the attributes and objects, background knowledge derived by bounded equational reasoning, and two experts, one searching for finite counterexamples with the Z3 SMT solver and one using the Vampire theorem prover, which also refutes implications with free magmas. Two explorations are discussed in detail, one of sixteen classical laws and one of all thirty-one one-variable laws with at most four operations. A user's guide closes the document.

Contents

1 Introduction **2**

2 Attribute exploration as implemented **3**
2.1 Overview . . . 3
2.2 Partial objects and the question asked . . . 3
2.3 Background knowledge . . . 4
2.4 Symmetries . . . 4
2.5 Experts that cannot always decide . . . 4
2.6 Resuming after a refuted unconfirmed implication . . . 5
2.7 Further details . . . 5

3 Exploring equational theories **6**
3.1 Attributes . . . 6
3.2 Objects: finite and free magmas . . . 6
3.3 Duality . . . 6
3.4 Background implications from bounded equational reasoning . . . 7
3.5 Background implications from the ETP . . . 7
3.6 The Z3 expert . . . 8
3.7 The Vampire expert . . . 8
3.8 Free magmas . . . 9
3.9 Seeding with known magmas . . . 10

4 Case study: sixteen classical laws **10**
4.1 Two explorations . . . 11
4.2 Where the experts differ . . . 11

5 Case study: one-variable laws of order at most four **12**
5.1 Background 12
5.2 Results 12

6 Case study: ETP laws of order at most two and three **13**
6.1 Choice of the laws 14
6.2 Order at most two 14
6.3 Order at most three 14

7 User's guide **16**
7.1 Installation 16
7.2 Describing the laws 16
7.3 Running an exploration 16
7.4 Choosing the expert and the time limits 16
7.5 Background knowledge 17
7.6 Reusing magmas 18
7.7 Reading the output 18
7.8 Using the library from Python 19
7.9 Practical remarks 19

1 Introduction

A *magma* is a set G with a binary operation $*$. An *equational law* (or *law*) is an equation $s = t$ between two terms built from variables with $*$, read as universally quantified: G *satisfies* $s = t$, written $G \models s = t$, if the two sides agree under every assignment of elements of G to the variables. Examples are commutativity $x * y = y * x$, associativity $(x * y) * z = x * (y * z)$ and idempotence $x = x * x$. The *order* of a law is the number of occurrences of $*$ in its two sides together.

The Equational Theories Project (ETP), initiated by Terence Tao in 2024, set out to decide, for every pair of laws E, F from a list of 4694 laws of order at most four, whether every magma satisfying E also satisfies F. Its laws are numbered, and we keep its numbers: Eq43 is commutativity, Eq4512 associativity, and so on.

From the point of view of Formal Concept Analysis, the natural object of study is the formal context

$$\mathbb{K}_{\mathrm{Mag}} = (\mathcal{G}, M, \models),$$

where M is a finite set of laws and $\mathcal{G}$ is the class of all magmas. Its object class is not even a set, but its implications are perfectly well defined: $P \to C$ holds in $\mathbb{K}_{\mathrm{Mag}}$ if every magma satisfying all laws of P satisfies all laws of C. By Birkhoff's completeness theorem, this is the case exactly when every law of C can be derived from the laws of P by the rules of equational logic. Its intents are the sets $\mathrm{Th}(G) \cap M$ for magmas G, the traces on M of equational theories. Attribute exploration is the natural tool for computing the canonical basis of $\mathbb{K}_{\mathrm{Mag}}$ and, with it, all its intents: an expert either confirms an implication, ideally by a proof, or refutes it by a magma.

The library implements the exploration algorithm of the book in a general form, in the Python package `conceptual_exploration`, and the equational-theories domain in the directory `explorations/equational_theories`. Section 2 describes how the general algorithm differs from the book. Section 3 describes the domain: attributes, objects, the duality symmetry, background knowledge, and the two experts. Sections 4, 5 and 6 discuss three explorations, and Section 7 explains how to run one's own.

Algorithm 1 The exploration loop (`AttributeExploration.run`)

Require: attributes M with a linear order, background implications $\mathcal{B}$, symmetries Φ, expert

$\mathcal{L} \leftarrow \mathcal{B}$ closed under Φ; $\mathcal{T} \leftarrow \mathcal{L}$ ▷ all implications; trusted ones
$P \leftarrow \mathcal{L}(\emptyset)$
loop
  $C \leftarrow P^\diamond \setminus P$ ▷ context closure, Section 2.2
  if $C = \emptyset$ **then**
    $P \leftarrow$ next $\mathcal{L}$-closed set after P; **continue**
  end if
  ask the expert about $P \to C$
  if the answer is an object g refuting $P \to C$ **then**
    add g and its images under Φ, completed with $\mathcal{T}$; **continue** with the same P
  else if the answer is "accept" **then**
    add $P \to C$ and its images to $\mathcal{L}$ (and to $\mathcal{T}$ if the expert decided it)
  else ▷ undecided: proved part $D \subseteq C$, partial object, flag
    add the partial object; add $P \to D$ to $\mathcal{L}$ and $\mathcal{T}$
    add $P \to C \setminus D$ to $\mathcal{L}$ only (unconfirmed), or record it as an open question
  end if
  if an object violates an implication of $\mathcal{L} \setminus \mathcal{T}$ **then**
    stop and resume (Section 2.6)
  end if
  $P \leftarrow$ next $\mathcal{L}$-closed set after P (or stop if there is none)
end loop

2 Attribute exploration as implemented

2.1 Overview

The exploration loop is the one of the book: the premises are the sets closed under the implications known so far, enumerated with NextClosure in lectic order, and for each premise P the expert is asked about $P \to C$, where C is what the current context suggests P implies. Four features change what happens around this loop:

1. objects are *partial*: for some attributes it may be unknown whether an object has them (Section 2.2);
2. a group of *symmetries* acts on the attributes, and implications and objects are added together with their images (Section 2.4);
3. the expert may *fail to decide* a question, in which case parts of the conclusion may be accepted without proof or left open (Section 2.5);
4. when an implication accepted without proof is later refuted, the exploration is *resumed* at the point where it went wrong (Section 2.6).

Algorithm 1 summarizes the loop; the notation is explained in the following subsections.

2.2 Partial objects and the question asked

An object g of the context is a *partial object*: a pair (g^+, g^-) of disjoint sets of attributes, those g is known to have and those it is known not to have. The attributes in neither set are unknown. An object refutes $P \to C$ if $P \subseteq g^+$ and $C \cap g^- \neq \emptyset$.

The conclusion offered to the expert is computed with the closure

$$P^\diamond = M \setminus \bigcup \{\, g^- : g \text{ an object with } P \subseteq g^+ \,\}.$$

An attribute belongs to $P^\diamond$ unless some object known to have all of P is known not to have it; $P^\diamond$ is thus the largest conclusion not refuted by the data at hand. When every row is complete, $P^\diamond = P''$ and the question is the one of the book. It is the cautious choice for incomplete data, and it is what makes partial rows useful: an object whose row is only partly known still removes from later conclusions the attributes it is known to lack.

In the equational-theories exploration, finite magmas always have complete rows (the expert evaluates every law on every counterexample), while free magmas (Section 3.8) may have unknown entries.

Completion. Every object is *completed* with the *trusted* implications $\mathcal{T}$ (background, confirmed, and their images; see Section 2.5): its positive side is replaced by its $\mathcal{T}$-closure, and every unknown attribute m whose addition would force an attribute of g^- is added to the negative side:

$$g^+ := \mathcal{T}(g^+), \qquad g^- := g^- \cup \{\, m : \mathcal{T}(g^+ \cup \{m\}) \cap g^- \neq \emptyset \,\},$$

the second step repeated until nothing changes. If the closure of the positive side meets the negative side, the object contradicts a trusted implication, and the program stops with an error: such an object can only come from a faulty expert. Objects are completed when they are added, and all objects are brought up to date whenever new trusted implications are accepted. Completion is what eventually fills in some unknown entries of partial rows.

2.3 Background knowledge

Background implications are given before the exploration starts, as in the book. They take part in the closure of premises and in the completion of objects, and they are reported with the source `background`. Without the features of the next subsections, the accepted implications form the canonical basis of the context relative to the background implications.

2.4 Symmetries

A *symmetry* ("mapping" in the code) is a map φ from the attributes to the attributes. When an implication $P \to C$ is accepted, so is its image $\varphi(P) \to \varphi(C)$ for every $\varphi \in \Phi$, unless the theory already entails it; images are reported with the source `mapped`. When an object g is added, so is, for every $\varphi \in \Phi$, a copy of g with the row

$$(\varphi^{-1}(g^+),\ \varphi^{-1}(g^-)),$$

the object that has an attribute m exactly when g has $\varphi(m)$. Copies are named $g\#k$, where k is the number of the mapping (the original is $g\#0$), and a copy whose row repeats one already added is skipped. Each implication and each object is mapped by each φ once and never by a composition, so Φ should be closed under composition, for instance a group. (In general a map may also send an attribute to a truth value; this is used in rule exploration, not here.)

Since the theory is closed under Φ at all times, the premise $\varphi(P)$ of the image of an accepted implication is not closed and never becomes a question: the expert is asked about one implication per orbit. The price is that the resulting set of implications, background, accepted and mapped together, is a basis of the theory, but in general not its canonical basis.

2.5 Experts that cannot always decide

The book assumes an expert who always answers correctly. The experts used here are programs with time limits, and a question may be left undecided. The library distinguishes four sources of implications:

`background` given in advance;

`confirmed` accepted by an expert who decided the question;

`unconfirmed` accepted without a decision: the expert found no counterexample but cannot rule one out;

`mapped` the image of another implication under a symmetry.

An expert reports that it has not decided a question it accepts through `is_conclusive()`. In addition, an expert may answer with an `Undecided` value, which carries

- a set $D \subseteq C$ of conclusion attributes it did prove: $P \to D$ is accepted as confirmed;
- optionally a partial object, which must not refute $P \to C$ and is added to the context (its unknown entries may be filled later by completion);
- a flag telling what to do with the rest $P \to C \setminus D$: it is accepted as *unconfirmed* if the expert knows nothing against it, or recorded as an *open question* if the expert knows it to be false without knowing which attribute fails.

Trusted and untrusted implications. Unconfirmed implications take part in the closure that generates the premises: they steer the enumeration as accepted implications do. They are, however, not used to complete objects, since they may be false and would then write false values into rows. Every object added later is checked against them; an object violating an unconfirmed implication refutes it.

Open questions. An open question is not added to the theory, and the enumeration continues with the next closed set as if the context had no question about P. The theory remains sound. It may, however, be incomplete at P: the expert knows that $P \to C \setminus D$ fails as a whole, but some of the attributes of $C \setminus D$ may still follow from P. Open questions are listed separately in the results.

2.6 Resuming after a refuted unconfirmed implication

When an object refutes an unconfirmed implication $P \to C$, all closed sets enumerated after P were computed with a false implication, and the enumeration from P on cannot be trusted. The exploration then stops and is resumed (`explore_in_rounds`): a new round starts with

- the implications accepted before $P \to C$, whatever their source;
- the *confirmed* implications accepted after it, which are true whatever steered the enumeration to them;
- every object found so far, and the open questions recorded before P;

and resumes NextClosure at P. Since NextClosure can continue from any set, and every closed set lectically before P was enumerated without the false implication, nothing before P is repeated. The false implication and the unconfirmed implications accepted after it are dropped; their questions are asked again if they arise. The final theory is sound and complete (up to open questions) but, because the confirmed implications kept were found under a different theory, it need not be a canonical basis.

2.7 Further details

- **Several questions per premise.** After a counterexample, the context closure of the same premise is recomputed and, if it is still larger than the premise, the expert is asked again, as in the book.

- **Continuing after an acceptance.** After $P \to C$ is accepted, the next closed set is computed from P under the extended theory. NextClosure reuses the step that produced P: the closure of P under the new theory is the next closed set if it adds no attribute lectically smaller than the one just added.
- **Rows of counterexamples.** With `evaluate_all`, the expert is given all attributes and returns the full row of a counterexample, not only its values on the question's attributes. The equational-theories exploration always does this.
- **Bit sets.** Attribute sets, rows and implications are stored as bit masks, and distinct rows are stored once with the objects sharing them; the context closure and the implication closure (a variant of Wild's algorithm) are integer operations.
- **Reading the result.** For display, `reduced_basis` rewrites the accepted implications: each premise is reduced to the attributes the rest of it does not entail, and each conclusion to the attributes that do not follow from the premise by way of the background, the other rules and all their images. A premise that entails every attribute is shown as concluding *everything*. The rewritten rules, read together with the background and the symmetries, entail exactly what the accepted implications do.

3 Exploring equational theories

3.1 Attributes

The attributes are the laws listed in a JSON file, each with an equation string and optionally a name and an ETP number:

```
[
  {"id": 43, "name": "Commutativity", "equation": "(x * y) = (y * x)"},
  {"id": 4512, "name": "Associativity", "equation": "((x * y) * z) = (x * (y * z))"},
  {"name": "Medial / Entropic", "equation": "((x * y) * (z * w)) = ((x * z) * (y * w))"}
]
```

Terms are parsed into trees with variables as leaves and $*$ as the only operation; laws are compared syntactically. The set of laws must be closed under duality (Section 3.3).

3.2 Objects: finite and free magmas

A *finite magma* is given by its Cayley table on $\{0, \dots, n-1\}$. Whether it satisfies a law is decided by evaluating both sides under all n^k assignments to the k variables of the law, so rows of finite magmas are always complete.

A *free magma* (Section 3.8) is a genuine magma, but usually an infinite one, so the library does not construct it; it represents it by its premise and computes only its row, which may be partial.

3.3 Duality

The *dual* of a term reverses all products, $(s * t)^{\mathrm{op}} = t^{\mathrm{op}} * s^{\mathrm{op}}$, and the dual of a law $s = t$ is $s^{\mathrm{op}} = t^{\mathrm{op}}$. The dual G^{op} of a magma has the transposed table, $a *^{\mathrm{op}} b = b * a$, and $G^{\mathrm{op}} \models E^{\mathrm{op}}$ if and only if $G \models E$. The map $E \mapsto E^{\mathrm{op}}$, with laws identified up to renaming variables and swapping sides, is the only symmetry used; it is an involution, so $\Phi = \{\,^{\mathrm{op}}\}$ is closed under composition. Counterexamples are thus added together with their duals, named `M`k and `M`k`^op` in the results, and implications together with their dual implications.

3.4 Background implications from bounded equational reasoning

For laws in a single variable, the library can derive background implications syntactically. A derivation uses three rules:

1. multiply both sides by the same term from the same side: from $s = t$ derive $u * s = u * t$ and $s * u = t * u$;
2. substitute a term for the variable: from $s = t$ derive $s[x := u] = t[x := u]$;
3. replace one occurrence of a subterm by an equal one: from $a = b$ and an equation e derive e with one occurrence of a replaced by b (or vice versa). This includes symmetry and transitivity.

These rules are complete for one-variable equational logic[1], but derivations may pass through arbitrarily large equations. Derivations are therefore bounded by an *order* N: all equations they pass through have order at most N. Within this bound, all rule instances are computed once, as Horn clauses between numbered equations, and the laws a set of laws derives are obtained by forward chaining.

A *transient order* $N' > N$ admits derivations of one more shape: a derived equation is multiplied or substituted into once, giving an equation of order at most N', and both sides of it are then rewritten independently with the derived equations, each rewrite replacing a side of a derived equation by the other side when that is no larger. Every pair of terms reached from the two sides that forms an equation of order at most N is derived. For example, substituting $x * x$ for x in $x * x = ((x * x) * x) * x$ gives an equation of order 10, which $x = (x * x) * (x * x)$ rewrites to $x = (x * (x * x)) * (x * x)$, of order 4. Such steps alternate with the forward chaining until nothing new is derived.

The derivation closure σ is a closure operator on sets of laws, and the background implications are computed from it in one of two ways:

- with a *premise size* K (default 2), every set P of at most K laws yields $P \to \sigma(P) \setminus P$, unless the implications kept so far entail it. The result is sound but may miss implications that need larger premises;
- with premise size 0, the canonical basis of σ is computed with NextClosure, which visits every σ-closed set and is much slower.

In both cases the background is sound: every background implication is valid in all magmas. Laws with several variables are not supported, since rule 2 would then have to rename variables apart and identify laws up to renaming.

3.5 Background implications from the ETP

The ETP has decided every implication between two of its 4694 laws, and publishes the outcomes as an implication graph[2]: a 4694×4694 array, row the premise and column the conclusion, run-length encoded. Together with the ETP's list of laws[3], it gives background implications for laws with any number of variables:

- every explored law is matched with an ETP law by a canonical form: its variables are renamed in the order of their first occurrence, for both orders of the two sides, and the smaller of the two strings is taken. Two laws have the same canonical form exactly when one becomes the other by renaming variables and swapping sides. A law whose `id` is not

[1] To check: every equational consequence of one-variable laws should be derivable as a chain of replacements of instances $u[x := v] \to w[x := v]$ of premise laws inside contexts, which rules 2, 1 and 3 provide in this order; this argument has not been written out in full.

[2] `https://teorth.github.io/equational_theories/implications/graph.json`

[3] `https://github.com/teorth/equational_theories/blob/main/data/equations.txt`

the ETP's number of the matched law stops the program with an error; laws the ETP does not list take no part;

- the laws that $x = x$ implies in the graph, which hold in every magma, form the conclusion of an implication with empty premise;
- every other listed law E yields $E \to \{F : E \text{ implies } F\}$, leaving out the laws valid in every magma.

The exploration then only asks about premises of several laws, since every implication with a single premise between listed laws is entailed by the background. The ETP's implications are proved in its Lean formalization[4], so this background is as trustworthy as the ETP; it is reported with the source `background`, like derived implications, and both kinds can be used together.

The script `etp.py` writes chosen ETP laws, together with their duals, as an equations file with the ETP's numbers as ids, so that the laws to explore can be chosen by number.

3.6 The Z3 expert

The `MagmaExpert` answers a question $P \to C$ as follows.

1. It checks a *pool* of magmas: 24 small standard magmas (projections, constants, cyclic groups, the two-element Boolean magmas, a rock–paper–scissors tournament, all two-element tables), every counterexample found so far, and any magmas given as seeds. The first magma satisfying P and violating a law of C is the counterexample.
2. Otherwise, for each size n from a minimum to a maximum (by default 1 to 6), it asks the Z3 SMT solver for a table of size n satisfying P and violating some law of C. The operation is an uninterpreted function restricted to $\{0, \ldots, n-1\}$, and the laws are grounded over all assignments. The first table found is the counterexample and joins the pool.
3. If no size yields a table, the implication is accepted. It is reported as confirmed if Z3 answered "unsatisfiable" for every size and as unconfirmed if it ran out of time (by default 30 seconds) for some size.

A "confirmed" implication thus only means that it has no counterexample of the searched sizes. It may be false, refuted only by a larger or an infinite magma, and both case studies show that this happens.

3.7 The Vampire expert

The `VampireExpert` uses the first-order theorem prover Vampire. A question $P \to C$ becomes a TPTP problem whose axioms are the universal closures of the laws of P and whose conjecture is the conjunction of the laws of C. Vampire then answers with a status: *Theorem* (it found a proof), *CounterSatisfiable* (the axioms do not imply the conjecture, shown by a finite model or by a saturated clause set), or a timeout.

The expert proceeds through the following steps and stops at the first one that decides the question.

1. **Pool and memory.** The pool is checked as by the Z3 expert. A question entailed by an implication proved before ($P' \subseteq P$, $C \subseteq C'$) is accepted, and an undecided answer is reused if the same question is asked again.

[4]To check: the graph's outcomes 3 and 7 (implication true) and 2 and 6 (false) are read as "explicitly or implicitly proved"; whether every such outcome is backed by a Lean proof was not checked.

2. **Proof.** Vampire's portfolio mode (`--mode casc`) runs on the problem (by default for 10 seconds). A proof confirms the implication. If the run instead ends with a finite model, the model is the counterexample.

3. **Quick model search.** Vampire's finite model builder (`--saturation_algorithm fmb`) looks for a counterexample of any size (by default for 1 second).

4. **Free magma.** The row of the free magma over P is computed, as described below. If a law of C is on its negative side, the free magma refutes the implication. If every law of C is on its positive side, the implication is proved.

5. **Per-law model search.** For each law c of C that the free magma did not prove, the model builder looks for a model of P violating c (by default for 10 seconds each, in parallel). This is much faster than a search for a model violating the conjunction C, which has to consider all the ways of violating it at once. Optionally, the model finder Mace4[5] searches for a model of P violating c at the same time (option `--mace4-time-limit`, off by default). Mace4 searches for tables size by size with its own propagation and symmetry reduction, and it sometimes finds models that Vampire's model builder does not reach in time.

6. **Undecided.** Otherwise, the laws of C that the free magma proved are accepted as confirmed. The free magma joins the context with its partial row. The rest of C is recorded as an open question if step 2 ended with CounterSatisfiable, that is, if Vampire showed $P \to C$ false as a whole, and is accepted as unconfirmed otherwise.

Every finite model is read from the output of Vampire or Mace4 and checked against the laws; a model that does not refute the question stops the program with an error. Vampire's proofs and its CounterSatisfiable verdicts obtained by saturation are trusted. The expert counts its answers by kind (`pool`, `proved`, `model`, `free magma`, `open`, `unconfirmed`), and the counts are printed at the end.

Unlike the Z3 expert, the Vampire expert never accepts an implication as confirmed without a proof, and it never accepts an implication it knows to be false.

3.8 Free magmas

Let P be a set of laws. Call two terms s and t, in countably many variables, *equivalent* if P derives $s = t$. The *free magma* F_P over P is the set of equivalence classes, with the operation $[s] * [t] = [s * t]$, which is well defined because P derives $s * t = s' * t'$ whenever it derives $s = s'$ and $t = t'$. It satisfies P, and by Birkhoff's theorem it satisfies a law L exactly when P derives L. Hence:

Proposition 1. *For every finite set M of laws and every $P \subseteq M$, the row of F_P is $\{ L \in M : P \vdash L \}$, the closure of P in $\mathbb{K}_{\mathrm{Mag}}$. In particular, F_P refutes every false implication with premise P.*

For one-variable laws, the submagma of F_P generated by one variable already has the same row. F_P is usually infinite, so it has no table. Its row is computed law by law. The laws that P implies by the background implications and by the implications Vampire has proved so far are on the positive side without asking. For each other law L, Vampire runs in portfolio mode on the problem "P implies L", all laws in parallel and the laws of the conclusion first. A proof puts L on the positive side, CounterSatisfiable on the negative side, and a timeout leaves L unknown. The laws of the conclusion get the proving time limit (by default 10 seconds); the others, which decide nothing about the question but complete the row, get a separate *row time limit* (option `--vampire-row-time-limit`, by default the proving time limit). With row time limit 0, only

[5] `https://www.cs.unm.edu/~mccune/prover9/`

the laws of the conclusion are asked about, and the other unknown laws stay unknown unless completion fills them in.

The row time limit matters when many questions end with a free magma that Vampire cannot fully decide: each of them then costs the row time limit for the laws outside the conclusion, which run in parallel on all cores. With the 149 laws of Section 6, a premise that implies few of them leaves well over a hundred laws to ask about, so that every such question costs about the row time limit times the number of laws divided by the number of cores: about 3 minutes on 8 cores with the default 10 seconds, and under 20 seconds with a limit of 1 second.

A free magma is a powerful object: when its row is complete, its intent is the closure of P in $\mathbb{K}_{\mathrm{Mag}}$, so it refutes every later question whose premise it satisfies and whose conclusion is not derivable. In the results, free magmas are named `F`k, the dual of F_P being the free magma over the dual laws of P. The negative side of a free magma rests on Vampire's CounterSatisfiable verdicts. Those obtained from a finite model could be checked; those obtained by saturation cannot be checked by the library and are trusted.

3.9 Seeding with known magmas

Magmas found in earlier explorations (the `magmas.json` of saved results) can be reused in two ways, and so can the finite magmas of the ETP: files with one table such as `[[0, 1], [1, 0]]` per line, after the word `Table` or an equation number, as in the ETP's `equational_theories/Generated/All4x4Tables/data` and `data/smallest_magma_examples.txt`, are read as well (repeated tables once). They can be added to the expert's pool, which is checked before any solver runs. Alternatively, they can be added to the context before the exploration starts: they are evaluated on every law, and objects whose rows the others account for are dropped (an object is redundant if, for each law it violates, another remaining object satisfies everything it satisfies and violates that law). No question that the seeded objects refute is then asked.

4 Case study: sixteen classical laws

The file `equations.json` lists the sixteen laws of Table 1. Some of them have two or more variables, so no background implications are derived; the only prior knowledge is the duality symmetry.

Law	Name	Equation
Eq2	Singleton	$x = y$
Eq3	Idempotence	$x = x * x$
Eq4	Left-zero	$x = x * y$
Eq5	Right-zero	$x = y * x$
Eq10	Left-central identity	$x = x * (y * x)$
Eq16	Steiner law 1	$x * (x * y) = y$
Eq25	Central identity	$x = (x * y) * x$
Eq26	Steiner law 2	$(y * x) * x = y$
Eq43	Commutativity	$x * y = y * x$
Eq323	Left-idempotent composition	$x * y = x * (x * y)$
Eq378	Right-idempotent composition	$(y * x) * x = y * x$
Eq4396	Left-alternative	$(x * x) * y = x * (x * y)$
Eq4435	Flexible	$(x * y) * x = x * (y * x)$
Eq4473	Right-alternative	$(y * x) * x = y * (x * x)$
Eq4512	Associativity	$(x * y) * z = x * (y * z)$
Medial	Medial (entropic)	$(x * y) * (z * w) = (x * z) * (y * w)$

Table 1: The laws of `equations.json`. The names are those of the file.

4.1 Two explorations

The laws were explored twice, once with each expert, with the default settings (Table 2).

	Z3, sizes 1–6	Vampire, 10 s / 1 s / 10 s
Questions	59	62
Implications accepted	27	27
of which unconfirmed	2	0
Implications in the theory (with images)	43	44
Counterexamples	32	35
Free magmas	—	0
Open questions	—	0
Running time	2 min 52 s	2 min 51 s

Table 2: The two explorations of the sixteen laws. Of Vampire's 35 counterexamples, 27 are models found by Vampire and 8 come from the pool. The Vampire time limits are those for a proof, the quick model search and the per-law model search.

The Vampire exploration proved every implication it accepted and refuted every other question with a checked finite magma, so its theory is exactly the set of implications between these laws that hold in all magmas. It is listed in reduced form in Table 3; each rule stands also for its dual, which swaps Eq4 and Eq5, Eq10 and Eq25, Eq16 and Eq26, Eq323 and Eq378, Eq4396 and Eq4473, and fixes the other laws.

1	Eq16, Eq26 → Eq43	15	Eq10 → Eq378
2	Eq4512 → Eq4435, Eq4473	16	Eq10, Eq323 → Eq4473
3	Eq26, Eq378 → Eq4	17	Eq10, Eq4435 → Eq25
4	Eq16, Eq378 → Eq10, Medial	18	Eq10, Eq4473 → Eq3
5	Eq323, Eq378, Medial → Eq4512	19	Eq10, Eq4396 → Eq323
6	Eq43 → Eq4435	20	Eq5 → Eq4512, Eq10, Eq16
7	Eq26, Eq43 → Eq16	21	Eq3, Medial → Eq4435
8	Eq43, Eq378 → Eq323	22	Eq3, Eq378 → Eq4473
9	Eq26, Eq4473, Medial → Eq4512	23	Eq3, Eq4473 → Eq378
10	Eq16, Eq4473, Medial → Eq4512	24	Eq10, Eq25 → Eq4435
11	Eq16, Eq4435, Eq4473 → Eq4396	25	Eq25, Eq4512 → Medial
12	Eq43, Eq4473 → Eq4396	26	Eq25, Eq43 → Eq2
13	Eq26, Eq4512 → Medial	27	Eq2 → everything
14	Eq43, Eq4512 → Medial		

Table 3: The implications between the sixteen laws, in the reduced form printed by the program. Together with their duals, they entail every implication that holds in all magmas.

4.2 Where the experts differ

The Z3 exploration accepted three implications that are false: one implication together with its dual, and one self-dual implication.

- Eq16, Eq4435, Eq4473 → Eq4396, Eq4512, Medial (and its dual). Only Eq4396 follows (rule 11). Associativity and mediality fail in a Steiner loop of order 10: a commutative loop in which every element is its own inverse, built from the Steiner triple system on nine points (the affine plane of order 3)[6]. It satisfies Eq16, Eq26, Eq43, Eq4396, Eq4435 and Eq4473 but none of the other laws.

[6] To check: the library only checked that the magma is a commutative loop with $x * x = e$ for its identity e and which laws it satisfies; the identification relies on the correspondence between Steiner loops of order $n + 1$ and Steiner triple systems on n points, and on the uniqueness of the system on nine points.

- Eq3, Eq16, Eq26, Eq43, Eq4435 → Medial. It fails in the Steiner quasigroup of order 7, the idempotent commutative quasigroup of the Fano plane, in which $x * y$ is the third point on the line through x and y[7].

Both counterexamples exceed the sizes the Z3 expert searches. Conversely, the two implications Z3 accepted as unconfirmed (Eq323, Eq378, Medial → Eq4396, Eq4435, Eq4473, Eq4512, rule 5 in reduced form, and Eq43, Eq4396, Eq4435, Eq4473, Eq4512 → Medial) are true, and Vampire proved them. Every implication Vampire accepted is entailed by the Z3 theory, as it should be.

The first of the false implications also illustrates the per-law model search of step 5. Before that step was added, the Vampire expert left this question undecided: the free magma over the premise decided neither associativity nor mediality in time, and a model violating their conjunction was not found within the time limit. The model builder finds the size-10 Steiner loop in a few seconds when it looks for a violation of a single law, while the search for a violation of both needs more than a minute and a half.

5 Case study: one-variable laws of order at most four

The file `equations_1var_order4.json` lists the 31 laws in one variable of order at most four, the trivial law Eq1: $x = x$ included. Up to the order of the sides, they are all the equations $s = t$ between the 23 one-variable terms with at most four operations such that the two sides together have at most four operations: 22 laws $x = t$, seven laws $x * x = t$, the law $x * (x * x) = (x * x) * x$, and Eq1. Since all laws have one variable, background implications can be derived.

5.1 Background

With order $N = 4$, transient order $N' = 10$ and premise size $K = 2$, the derivation closure yields 111 background implications. With premise size 0, the canonical basis of the derivation closure has 95 implications; the two sets entail each other, so for these laws premises of size two lose nothing. The 111 implications have 108,635 closed sets.

5.2 Results

Table 4 compares two Vampire explorations, which differ only in the order of the steps of the expert, with two explorations by the Z3 expert.

No implication beyond the background. Both Vampire explorations refuted every question: no implication was accepted, and the theory is the background itself. This is a mathematical result: *every implication between one-variable laws of order at most four that holds in all magmas follows from the 111 background implications*, that is, it can be derived by equational reasoning through equations of order at most four, with single steps reaching order ten, from premises of at most two laws. Conversely, every non-implication is witnessed either by a finite magma, checked by the library, or by a free magma whose relevant negative entry rests on Vampire's saturation. The final context of the second run has 37,177 objects (originals and duals) with pairwise distinct rows, and its 108,635 intents are exactly the closed sets of the background. Every row was decided completely: no free magma has an unknown entry.

The counterexamples. The finite counterexamples found in the first Vampire run have the following sizes (originals only):

[7] To check: the library only checked which laws the magma satisfies; the identification relies on idempotent totally symmetric quasigroups being Steiner quasigroups and on the uniqueness of the Steiner triple system on seven points.

	Vampire		Z3, sizes 1–6	
	full search first	quick search first	no background	background
Background implications	111	111	—	103
Questions	17,835	18,655	8,189	8,342
Implications accepted	0	0	887	842
of which false	0	0	837	837
Refuted by a finite model	15,894	14,688	7,302	7,500
Refuted by a free magma	1,941	3,967	—	—
Open or unconfirmed	0	0	0	2
Closed sets of the theory	108,635	108,635	75,127	75,127
Running time	8 h 05 min	2 h 25 min	not recorded	3 h 18 min

Table 4: Explorations of the 31 one-variable laws. "Full search first" runs a 10-second finite model search on the conjunction before computing the free magma; "quick search first" runs a 1-second search, then the free magma, then a 10-second search per law. "Refuted by a finite model" includes the 7 questions answered by the pool. The Z3 exploration with background used order 7 and no transient order. The Vampire running times are wall-clock times on an 8-core laptop; the second excludes a 40-minute sleep of the machine.

Size	2	3	4	5	6	7	8	9	10
Magmas	5	92	876	2,741	2,837	2,247	4,694	2,200	202

Most counterexamples are larger than six elements, and some implications have no finite counterexample that Vampire finds in time; for those the free magma is the counterexample. A typical example is question 685 of the second run, whose premise consists of Eq1 and seven laws including $x = (((x * x) * x) * x) * x$ and $x * x = x * (x * (x * x))$: the free magma over the premise satisfies exactly the eight premise laws and refutes all 23 others at once.

What the Z3 explorations got wrong. The Z3 expert searches only up to six elements, and its explorations accepted 887 and 842 implications. Only 50 and 5 of them, respectively, follow from the background (the second exploration already had most valid implications as background); the remaining 837 are false in both cases. The false implications made the enumeration skip about a third of the closed sets, which is why the Z3 explorations asked fewer than half as many questions. In this domain, bounded finite-model search is a poor expert: an implication without small counterexamples is more often false than true.

Order of the steps. In the first Vampire exploration, a 10-second model search on the conjunction preceded the free magma. Most questions that ended with a free magma thus first spent 10 seconds in a hopeless search. Running a 1-second search first, the free magma next, and a 10-second search per law last cut the running time by a factor of more than three. The number of questions differs between the runs because different counterexamples were found, which changes the context and therefore which closed sets need a question; the theory is the same.

6 Case study: ETP laws of order at most two and three

The ETP has decided every implication $E \to F$ between two of its laws, but an implication with a larger premise, such as $E_1, E_2 \to F$, is not determined by those: E_1 and E_2 may together imply a law that neither implies alone. Attribute exploration with the ETP graph as background (Section 3.5) asks exactly about such premises. The two explorations of this section apply it to the ETP laws of order at most two and three.

6.1 Choice of the laws

Laws that imply each other are interchangeable in an implication, so it suffices to explore one law from each class of equivalent laws. The ETP lists 46 laws of order at most two, in 23 classes, and 410 laws of order at most three, in 146 classes. One law is chosen from each class: the law with the smallest number, or, if the class contains a law equal to its own dual, the first such law. If no law of a class is self-dual, the dual of the chosen law, which lies in the dual class (possibly the same class), is chosen as well and represents that class, so that the set is closed under duality. This gives 24 laws of order at most two (`equations_order2.json`) and 149 of order at most three (`equations_order3.json`). Both files were written by `etp.py`, which computes the classes from the ETP graph and lists the members of each representative's class in the file:

```
python explorations/equational_theories/etp.py equations.txt \
    --max-order 3 --graph graph.json -o equations_order3.json
```

Each set includes Eq1: $x = x$ and Eq2: $x = y$, which hold in every magma and only in the trivial magma, respectively. The two explorations took 18 and 138 background implications from the ETP graph.

6.2 Order at most two

With the Vampire expert and the ETP background, the exploration of the 24 laws asked 45 questions. It found 19 counterexamples, 6 of them two-element magmas of the pool and 13 found by Vampire, of sizes 3 to 5, and proved 26 implications; nothing was left unconfirmed or open, and no free magma was needed. The theory has 59 closed sets, and each of the 26 rules of its reduced basis has a premise of two laws. For example:

- $\text{Eq25}, \text{Eq43} \to \text{Eq2}$: a commutative magma satisfying $x = (x * y) * x$ is trivial;
- $\text{Eq26}, \text{Eq29} \to \text{Eq43}$: the laws $x = (x * y) * y$ and $x = (y * x) * y$ together imply commutativity;
- $\text{Eq25}, \text{Eq8} \to \text{Eq3}$: the laws $x = (x * y) * x$ and $x = x * (x * x)$ together imply idempotence.

Since every question was decided, this is the complete implicational theory of the ETP laws of order at most two: every implication between them that holds in all magmas follows from the 26 rules, the 18 single-premise implications of the ETP graph, duality, and the equivalence of the laws of each class.

6.3 Order at most three

The 149 laws have far more closed sets, and many premises combine laws whose consequences Vampire finds hard to decide. Two explorations were run, both with the ETP background, the context seeded with the ETP's finite magmas (Section 3.5; 233 of 1374 tables were kept), a row time limit of 1 second, and otherwise the default limits. The second also used Mace4 in the per-law model search, for 30 seconds per law. Table 5 compares them.

Where Vampire gets stuck. Most hard questions involve Eq255: $x = ((x * x) * x) * x$ or its dual Eq47: $x = x * (x * (x * x))$, combined with laws in two or more variables. In the second exploration, 44 of the 48 unconfirmed implications and all five open questions have one of these two laws in their premise. Their premises are often large: the open questions have 16 to 18 laws in their premise.

	Vampire	Vampire and Mace4
Questions	1,310	1,326
proved	736	746
refuted by a finite model	437	440
refuted by a free magma	75	76
unconfirmed	57	59
open	5	5
Confirmed implications	750	754
Unconfirmed implications in the theory	57	48
of which shown false afterwards	6 or more	—
Rules of the reduced basis	793	795
Closed sets of the theory	3,650	3,663
without the unconfirmed implications	8,378	8,377
Rounds	1	2
Running time	2 h 48 min	4 h 34 min

Table 5: Explorations of the 149 representatives of the ETP laws of order at most three, with the ETP graph as background and the ETP's finite magmas in the context. The counts of unconfirmed implications do not include their duals. The running times are wall-clock times on an 8-core laptop; the second excludes a sleep of the machine of about an hour.

Testing the unconfirmed implications. After the first exploration, each law of the conclusion of each of its 19 unconfirmed questions (109 laws in all) was tried again, separately, with longer limits:

- Vampire, 60 seconds in portfolio mode and 60 seconds with its model builder, first on the premise alone and then with the one-variable laws that the premise derives (Section 3.4) added as lemmas: none of the 109 laws was decided;
- Prover9, 60 seconds: no proof;
- Mace4, 60 seconds: models of size 16 violating 34 of the laws, found within 0.2 to 31 seconds. These settled six of the 19 questions as false, all their unproved laws being violated, and three magmas sufficed for all six.

So at least six of the 57 unconfirmed implications of the first exploration are false, and the enumeration, which they steered, skipped closed sets. Its 3,650 closed sets are therefore not those of the true theory.

The second exploration. With Mace4 in the per-law search, the expert found the counterexamples of size 16 during the exploration itself, together with models of sizes 11 and 13 that Vampire had missed (17 tables in all, counting duals). One of them refuted an unconfirmed implication accepted 260 questions earlier; the exploration resumed at that question (Section 2.6), which made it run in two rounds. In the end, 48 unconfirmed implications and 5 open questions remain, and 46 of the 48 have the same premise as an unconfirmed implication of the first exploration. No counterexample to them is known, but they are not proved either.

What the results say. The 754 confirmed implications, together with the background and their duals, are proved. Every non-implication witnessed by a finite magma is checked, and those witnessed by free magmas rest on Vampire's saturation. The 3,663 closed sets of the theory are correct if the 48 unconfirmed implications are true. Without them, the theory has 8,377 closed sets, an upper bound on the number of closed sets of the true theory, since each confirmed implication holds. Settling the unconfirmed implications and the open questions needs either proofs that Vampire and Prover9 do not find within a minute, or counterexamples

that Vampire and Mace4 do not find: finite magmas beyond their reach, or free magmas whose rows Vampire cannot decide.

7 User's guide

7.1 Installation

The library needs Python 3.11 or later. In a clone of the repository:

```
pip install -e ".[examples]"     # the library, Z3 and Jupyter
```

The Vampire expert additionally needs the `vampire` executable on the path, for example via Homebrew (`brew install vampire`) or a release from `https://vprover.github.io`. The explorations described here used Vampire 5.1.0. The optional Mace4 search needs the `mace4` executable, which comes with Prover9 (`brew install prover9`).

7.2 Describing the laws

Write the laws to a JSON file as in Section 3. Every entry needs an `equation` string; `name` and `id` are optional, and laws without an id are identified by their name or their text in the results. Terms use variables (single letters or identifiers), `*` and parentheses; an unparenthesized product associates to the left, so `x * y * z` is read as $(x * y) * z$. The set must be closed under duality: for every law, a law equal to its dual up to renaming variables and swapping sides must be in the file, or the program stops with an error naming the missing dual. Derived background implications (Section 3.4) require all laws to have the same single variable; background from the ETP (Section 3.5) does not. To explore ETP laws chosen by number, let `etp.py` write the file:

```
python explorations/equational_theories/etp.py equations.txt 3 43 4512 \
    -o my_laws.json
```

where `equations.txt` is the ETP's list of laws; the duals of the chosen laws are added. With `--max-order K --graph graph.json` instead of numbers, it writes one law of each class of equivalent ETP laws of order at most K, closed under duality (Section 6), each with the members of its class.

7.3 Running an exploration

From the root of the repository:

```
python explorations/equational_theories/explore.py \
    --equations my_laws.json --expert vampire
```

Table 6 lists the options.

7.4 Choosing the expert and the time limits

- Use the **Vampire expert** when the result should be correct. Its confirmed implications are proved, and its counterexamples are checked magmas or free magmas. If it accepts something as unconfirmed or leaves a question open, the summary says so.
- Use the **Z3 expert** for a quick first impression, or when Vampire is not available. Treat its accepted implications as conjectures: as the case studies show, they may be false.

Option	Default	Meaning
`--equations PATH`	`equations.json`	the laws to explore
`--expert z3|vampire`	`z3`	which expert answers the questions
`--vampire-prove-time-limit S`	10	seconds for a proof, and for each law of the conclusion in a free magma's row
`--vampire-row-time-limit S`	prove limit	seconds for each other law of a free magma's row; 0 leaves them unknown
`--vampire-quick-model-time-limit S`	1	seconds for the first finite model search
`--vampire-model-time-limit S`	10	seconds for each per-law finite model search
`--mace4-time-limit S`	0	seconds for Mace4 in each per-law search; 0 leaves Mace4 out
`--min-search-size N`	1	smallest table size Z3 tries
`--max-search-size N`	6	largest table size Z3 tries
`--z3-timeout-ms MS`	30000	Z3 time per size; 0 means unbounded
`--background-order N`	off	derive background implications through equations of order at most N (at least the largest order of a law)
`--background-transient-order M`	N	let single steps reach order M
`--background-premise-size K`	2	largest premise of a background implication; 0 computes the canonical basis
`--etp-graph PATH`		the ETP's implication graph, as background; needs `--etp-equations`
`--etp-equations PATH`		the ETP's list of laws, to match the explored laws
`--seed-magmas PATH...`		add magmas to the expert's pool
`--seed-context PATH...`		add magmas to the context before starting
`--report-every N`	1	print every Nth question; 0 prints none
`--results-dir PATH`	new directory	where to save the results

Table 6: Options of `explore.py`. The Z3 options apply to the Z3 expert only, and the Vampire options to the Vampire expert only.

- Longer **proving time limits** help when implications are true but hard to prove. The laws of the conclusion in a free magma's row get the same limit, and a law Vampire cannot decide in time stays unknown.
- A short **row time limit**, such as 1 second, speeds up explorations with many laws, where most of a free magma's row is outside the conclusion. The price is more unknown entries, so a free magma refutes fewer later questions by itself.
- Longer **model time limits** help when counterexamples are large. The per-law search usually finds a model faster than the search on the whole conclusion. Adding Mace4 with `--mace4-time-limit` costs up to that many seconds for every question that reaches the per-law search, which is worth it when Vampire's model builder misses counterexamples of moderate size, such as the magmas of size 16 that refuted several unconfirmed implications between laws of order at most three.
- Vampire runs in parallel on all cores when it computes a free magma's row or searches for models law by law, so other work on the machine slows the exploration down.

7.5 Background knowledge

For one-variable laws, background implications usually pay off: they are computed in seconds and remove the questions whose answer is a derivation. Start with `--background-order` equal

to the largest order of the laws and a transient order of about twice that; the 31-law exploration used 4 and 10. A larger order finds more implications at a cost that grows quickly. A premise size of 2 is usually enough; compare with `--background-premise-size 0` on a small set of laws if in doubt.

For laws the ETP lists, pass its implication graph and its list of laws:

```
python explorations/equational_theories/explore.py \
    --equations my_laws.json --expert vampire \
    --etp-graph graph.json --etp-equations equations.txt
```

This background costs nothing to compute and leaves only questions whose premise has several laws. It helps most when the implications between pairs of laws are hard to settle.

7.6 Reusing magmas

To start a new exploration with the magmas of an earlier one, pass its `magmas.json` to `--seed-context` (fewer questions) or to `--seed-magmas` (the same questions, answered faster). Free magmas are not reused. The ETP's files of tables can be passed in the same way. Seeded into the context of the exploration of Section 6, the 1374 distinct tables of `All4x4Tables` and `smallest_magma_examples.txt` were reduced to 233 magmas.

7.7 Reading the output

The program prints every question with its answer and writes the same text to `exploration.log`:

```
Number of implications: 111
Number of objects: 1358
Question 685: Eq1: x = x, Eq1629: ... -> Eq8: x = (x * (x * x)), ...
Counterexample found: Free magma over {Eq1, Eq1629, Eq1832, ...}
Satisfies: ...
Does not satisfy: ...
```

The counts are those before the answer; the objects include the duals. An accepted question is followed by `Confirmed` or `Unconfirmed`, and an undecided one by `Open` or `Unconfirmed` together with the laws it proved. At the end, the program prints a summary (questions, accepted implications, counterexamples, open questions, rounds, and the Vampire expert's answers by kind), the reduced basis, any open questions and refuted unconfirmed implications, and a few counterexamples.

The results directory contains:

`exploration.log` everything printed;

`implications.txt`, `implications.json` the whole theory, every implication with its source (`background`, `confirmed`, `unconfirmed`, `mapped`), followed by the refuted unconfirmed implications and the open questions;

`magmas.json` the finite magmas of the context with their tables, under the names Mk and Mk`^op`;

`free_magmas.json` the free magmas Fk, each with its premise and its known positive and negative laws;

`context.cxt` the objects with complete rows, in Burmeister's format;

`partial_context.cxt` all objects in the same format, with `?` for unknown entries.

The program saves the results only at the end. If it is interrupted, only the log remains.

7.8 Using the library from Python

The runner is a thin layer over the library, which can be used directly:

```
from conceptual_exploration import explore_in_rounds, reduced_basis, report_every
from explorations.equational_theories.magma import ETP
from explorations.equational_theories.vampire import VampireExpert
from explorations.equational_theories.background import background_implications

laws = ETP.load_equations("my_laws.json")
expert = VampireExpert(attributes=laws, prove_time_limit=10,
                       quick_model_time_limit=1, model_time_limit=10)
result = explore_in_rounds(
    laws, expert,
    background_implications=background_implications(laws, 4, 2, 10),
    mappings=[ETP.get_duality_mapping(laws)],
    evaluate_all=True,
    on_question=report_every(100),
)
for rule in reduced_basis(result.base):
    print(rule)
print(result.base.open_questions, expert.answers)
```

`run_magma_exploration` in `explore.py` does the same with the options above and saves the results; the notebook `explore.ipynb` shows smaller examples. Another expert can be plugged in by subclassing `Expert` and implementing `validate`, which returns a refuting `PartialObject`, `None` to accept, or an `Undecided` value.

7.9 Practical remarks

- **Size.** The number of questions grows with the number of closed sets of the final theory, which may be large: 108,635 for 31 laws without a single accepted implication. Expect hours for such an exploration and minutes for one with a few dozen closed sets per law.
- **Long runs.** Keep the terminal open. Putting the machine to sleep is harmless in principle, but a Vampire run in progress at that moment may time out on waking, since the time limits are wall-clock times; this can at worst turn a refutation into an undecided answer, which the summary reports.
- **Reproducibility.** Equations are sorted before they are given to the solvers, so a Z3 exploration is reproducible. Vampire's time limits make a Vampire exploration depend on the machine's speed: a different run may find different counterexamples and ask a different number of questions, but the theory is the same unless a question is left undecided.
- **Trust.** Finite counterexamples are checked by the library. Proofs and saturation-based CounterSatisfiable verdicts are Vampire's; independent confirmation would require checking Vampire's proofs or the free magmas' negative entries separately.